STRAYDOG SYNDICATIONS · SECOND STORY INITIATIVE

The AISE Manifesto

Ten tenets for AI-Integrated Software Engineering, and why the returned citizen is built for it.

"I learned to code inside. I learned to think like an engineer outside. This is the document I wish someone had handed me on day one of both."

ERIC "HUNTER" PETROSS

Second Draft · August 2026 · Circulated for community review

AI Orchestrator Curriculum

ONE

Preamble

The Last Mile opened the door. They put a curriculum and a compiler in front of people the world had already filed away, and they bet that what was inside those walls was worth developing. That bet is the reason I am writing anything at all.

What TLM gave me wasn't only technical. It was cadence. Show up, week after week. Mentor the person a cohort behind you while the person a cohort ahead pulls you forward. That structure taught me more about how engineering teams actually work than any syllabus could have.

The cadence is what carried me through Code the Dream, where the scaffolding was gone and the studying was mine to organize. The habit transferred cleanly. It is the only reason I finished the bootcamps that came after.

Columbia Justice Through Code, now Fair Chance Futures, gave the work institutional weight and a national stage. None of these programs built halfway houses for talent. They built launchpads. This document sits on infrastructure that other people constructed, and I would rather say so at the top than bury it in an acknowledgment at the back. That training also pointed me back to where I started. I teach MERN stack at The Last Mile now, in the same program that changed my trajectory.

What follows is my attempt to name the thing all three programs were pointing at, before any of us had language for it.

TWO

What AISE Is

> **AI-Integrated Software Engineering.** The practice of building software with machine intelligence as a working component of the process, rather than a novelty bolted on at the end of it.

Prompt engineering is one narrow skill inside AISE, not a synonym for it. The discipline is the union of four things, held tightly enough that you can trace how data moves through all of them at once:

- **Full-stack fluency.** You cannot direct a build you could not have written yourself, slowly, given enough time.
- **Working knowledge of model behavior.** Not the math. The failure modes, the drift, the confident wrong answer and how to spot it early.
- **Deliberate context design.** Framing, constraints, and an explicit definition of done, authored before the first query.
- **Infrastructure awareness.** Where the data goes, what it costs, and what breaks when the vendor changes the terms.

The practitioner's job is orchestration. Decide what to build. Direct the machine through the build. Hold the standard for what finished means, because the machine has no opinion about that and never will.

Ten tenets follow. They are not a certification track and there is no test at the end. They are the operating assumptions I work from, written down so the person coming up behind me doesn't have to reverse-engineer them from scratch the way I did.

THREE

The Ten Tenets

Tenet 01

The machine doesn't know. It approximates.

The first thing TLM taught me was how to read a stack trace without panicking. The first thing the models taught me was that they are not an authority. A model is a fast, extraordinarily well-read collaborator that has never once left the library. More text processed than any human alive, and zero lived experience to anchor a word of it.

That's the opening, not the flaw. Every output is a best guess at what you asked for. Once that lands, you stop deferring and start directing. I quit treating the model like a search engine, started treating it like a junior developer I was pairing with, and the work got sharper inside a week. The tool never changed.

Tenet 02

You're not prompting. You're negotiating.

Inside, you negotiate everything. Housing. Access to anything scarce. How to put a request to someone who holds the answer and owes you nothing. You read the room, you frame the ask, you handle the objection before it arrives. Call that manipulation if you want to. I would call it precision communication under constraint, and most people never have to build the muscle.

Working with a model runs on the same muscle. Most people use it like a vending machine: insert request, collect output, complain about the snack. Your first query is not the deliverable. It's an opening position. Everything between that and production is the alignment loop, and if you came up the way I came up, you already know how to run one.

Tenet 03

Context is infrastructure.

Clean code is a communication practice. You write for the next developer, not the compiler. The same law governs AI-integrated work, and it is unforgiving: what you get back is a direct function of what you built going in. So I write prompts the way I write commit messages that matter. What am I asking for, why does it matter, what does done look like. Domain framing, hard constraints, explicit success criteria. That habit alone puts you ahead of most people currently touching this technology, which says more about the state of the field than it does about me.

> *"Every program added a layer. Not because the last one was incomplete, but because mastery isn't a destination you arrive at. It's a posture you hold. You keep building. That isn't a tech industry value we had to be taught. We brought it in with us."*
>
> TLM ALUMNUS · CODE THE DREAM MENTOR · FAIR CHANCE FUTURES COHORT

Tenet 04

Iteration is the method, not evidence of failure.

I used to think a follow-up question meant I had gotten the first one wrong. Code review cured that. Every pull request was a conversation, not a verdict.

The models teach the same lesson from another angle. The first response is a draft. The second is a refinement. The fifth exchange is usually where the actual work happens. I have shipped production features out of sessions that looked, from the outside, like a man arguing with a chat window for three hours. Don't skip the reps.

Tenet 05

The skill is orchestration, not replacement.

People ask me constantly whether AI is going to take developer jobs. I understand the fear. We know precisely what it feels like to be made redundant by a system that never consulted us on the decision, and I am not going to wave that away with a statistic.

Here is what I actually observe. An unskilled operator with a model produces unskilled output, faster. A practitioner with the same model produces work that used to require a full team. The differentiator was never the model. It is the person running the session, the one who knows what the thing is supposed to sound like when it's right. Nobody has automated that, and I have not seen a credible attempt.

Tenet 06

Systems thinking is the core competency.

Each program taught a different slice. Web development at TLM. Practice and community at Code the Dream. Career architecture and professional network at Fair Chance Futures. I experienced them as three separate things. They were building the same faculty from three sides: the ability to hold a whole system in view while working on one piece of it.

That is the entire line between someone poking at a black box and an engineer. Commit to the whole picture even on the days you can only touch one corner of it.

Tenet 07

What was taken can become leverage.

Incarceration took years. It took proximity to my family. It took the unremarkable career momentum that people who never stopped working don't know they are carrying. I am not going to dress that up. The losses were real, and some of them do not come back.

Here is what it did not take. The ability to read a room fast. To operate on almost nothing. To build trust from zero in an environment where trust is the scarcest resource on the tier.

Recruiters file those under soft skills. They are systems skills, and they transfer. A returned citizen walking into a technical interview is carrying years of constraint-based problem solving that most computer science graduates have never been forced to develop. Learn to name it that way in the room. It is accurate, and it is defensible, and it reframes the entire conversation you were bracing for.

Tenet 08

Zero-cost tooling is a discipline.

The AI Orchestrator curriculum runs on no paid tools. That is deliberate, and the reason is not that free tools are merely adequate. It's that expertise built on free tools cannot be revoked when somebody upstream changes a pricing tier.

When I stood up my first local stack, models running on my own hardware with no cloud dependency and no API bill, I learned the architecture in a way that pointing a browser at a hosted dashboard would never have taught me. The constraint was the instructor. We already know how to learn under constraint. The Digital Cliff is real, and it is also crossable, and the crossing does not require a credit card.

Tenet 09

You build for the people behind you.

I went back to teach at The Last Mile because someone came back for me. There is no philosophy underneath that. The pathway exists because people who finished it turned around and held the door.

Curriculum you contribute, repositories you open, hours you spend mentoring: that is infrastructure, not charity. The second-chance technology pathway was not handed down by institutions. It was built by practitioners, one returned engineer at a time, and it stays standing the same way it went up. When you level up, bring it back. That is the covenant, and it is also just how the ecosystem survives long enough to matter to anyone.

Tenet 10

Nobody is waiting for permission.

I didn't wait for the industry to rule on whether I was hireable. I built while the decision was pending. I built while people questioned out loud whether someone with my record belonged in the room. I built through every version of the conversation where the subtext was *maybe not you.*

The AI transition is not waiting on us either, and it is not asking whether we would like to be included. The only live question is which side of the build we are standing on. AISE isn't a certification. It's a decision to engage the technology as a practitioner, starting now, with whatever is currently in your hands. No degree. No prior access. Curiosity, discipline, and a connection. Everything after that is reps.

I'm proof the reps work. So are you, if you're still reading.

FOUR

A Declaration for the Returned

I remember typing my first function inside. I had no idea what I was doing. I knew something had opened.

TLM didn't just teach me JavaScript. It handed me a way of thinking I hadn't known was missing: take a complex problem, decompose it, build toward a solution out of the pieces actually in front of you. That is a coding skill. It is also, and I mean this without any inspirational framing, a survival skill. It works on sentences and it works on years.

Code the Dream gave the practice a community. Fair Chance Futures, built by the team at Columbia Justice Through Code, gave it institutional weight and a seat at a table that historically would not have set a place for us. Every program stacked on the one before it. Every instructor and program director who decided this work was worth doing added a layer to the ground I am standing on right now.

This document is what that investment produced. Not a thank-you note. A proof of concept. A TLM alumnus who became an instructor. A student who became a mentor. A returned citizen writing curriculum, shipping code, running his own AI infrastructure, and teaching the next cohort the thing nobody handed me a map for.

I am inside the AI transition rather than watching it from the shoulder, and I am documenting the methodology while it is still forming, so that the next person does not have to work it out alone. That is the entire reason this exists.

To the founders and directors reading this: you will recognize what is here. It is the thing you were building toward. Thank you for believing it was possible before any of us could demonstrate it.

You do not need a clean record. You need clean code. The machine will meet you where you are and it does not run a background check. It only knows the quality of what you bring to the conversation. So bring all of it. Every hard lesson, every systems insight, every hour of deliberate practice.

That is AISE. Come build it.

The future is not a locked door.
It's an unfinished build.
Get in the repository.

SIGNED

Colophon

Eric “Hunter” Petross

Founder — StrayDog Syndications LLC · Second Story Initiative

MERN Stack Instructor — The Last Mile

Python Mentor — Code the Dream

Alumnus — The Last Mile · Code the Dream · Fair Chance Futures / Columbia Justice Through Code

---

METHODOLOGY NOTE

This document was produced the way it argues you should work. It was drafted, argued with, and revised across many sessions with Claude (Anthropic), with orchestration support from Cortana, a personally trained open-source agent running on the StrayDog Kennels Stack (HK-888).

Naming that is not a disclaimer. It is the demonstration. A manifesto about human-machine collaboration that concealed its own would be arguing against itself on the page. The tenets are mine. The judgment calls are mine. The reps are documented.

SECOND DRAFT · AUGUST 2026 · CIRCULATED FOR COMMUNITY REVIEW
AI ORCHESTRATOR CURRICULUM · STRAYDOG SYNDICATIONS LLC